

Interação com APIs

05

Manipulação de Dados com APIs

Criando recursos com POST no Angular



Situação de Aprendizagem

O Desafio da Aula

Uma empresa está desenvolvendo um **sistema de cadastro corporativo de tarefas**. O usuário poderá cadastrar novas atividades informando título, descrição, prioridade, categoria, data limite e status inicial. Ao clicar em **Salvar**, os dados deverão ser enviados para uma API.

❓ **Pergunta-problema:** Como transformar dados digitados pelo usuário em uma requisição HTTP capaz de criar um novo recurso no servidor?

 Título Nome da tarefa	 Descrição Detalhes da atividade	 Prioridade Alta, média ou baixa
 Categoria Classificação da tarefa		 Status Estado inicial da tarefa

Contextualização Profissional

No mercado de trabalho, o front-end vai muito além de exibir informações — ele é responsável por **criar e persistir dados no servidor**. Toda ação do usuário que gera um novo registro depende de uma requisição POST bem estruturada.



Cadastro de Cliente

Novos clientes registrados no sistema de CRM da empresa



Envio de Pedido

Pedidos e-commerce enviados ao backend de processamento



Criação de Conta

Registro de novos usuários com validação e persistência



Abertura de Chamado

Tickets de suporte enviados ao sistema de helpdesk



Criação de Tarefa

Atividades corporativas enviadas ao gerenciador de projetos



Registro de Pagamento

Transações financeiras registradas com segurança via API

Front-end não apenas consulta dados — ele também cria informações no servidor.

Relembrando o Método POST

O que é POST?

O método HTTP **POST** é utilizado para **criar novos recursos** no servidor. Ao contrário do GET, que apenas solicita dados, o POST envia informações no corpo da requisição para que o servidor processe e persista um novo registro.

- ✔ POST cria um novo recurso. O servidor responde com **201 Created**.

Características principais



Envia body

Os dados viajam no corpo da requisição, não na URL



Payload complexo

Pode conter objetos aninhados, arrays e estruturas ricas



201 Created

Resposta padrão indicando que o recurso foi criado com sucesso



Mais seguro que GET

Dados não ficam expostos na URL da requisição

O que é um Payload?

O **payload** é o corpo da requisição HTTP — o conjunto de dados que o front-end envia ao servidor para criar ou atualizar um recurso. Ele define o **contrato de dados** entre a aplicação cliente e a API.

Exemplo de payload da nossa tarefa

```
{
  "titulo": "Estudar Angular",
  "descricao": "Consumir APIs com HttpClient",
  "prioridade": "alta",
  "categoria": "estudo",
  "status": "pendente",
  "data": "10/10/2026"
}
```

Conceitos do payload

1

Estrutura

Organização dos campos em pares chave-valor no formato JSON

2

Serialização

Conversão do objeto TypeScript em texto JSON para transporte via HTTP

3

Contrato da API

Os campos devem corresponder exatamente ao que o servidor espera receber

Criando a Interface Tarefa

No Angular, utilizamos **interfaces TypeScript** para modelar os dados que serão enviados à API. Isso garante tipagem estática, segurança em tempo de desenvolvimento e um contrato claro entre camadas da aplicação.

interface Tarefa — model/tarefas.ts

```
export interface Tarefa {  
  id?: number;  
  titulo: string;  
  descricao: string;  
  prioridade: string;  
  categoria: string;  
  status: string;  
  dataLimite: string;  
}
```

Por que usar interfaces?

1

Tipagem

O compilador alerta sobre campos inválidos ou ausentes

2

Segurança

Erros são detectados em tempo de desenvolvimento, não em produção

3

Autocomplete

A IDE sugere campos disponíveis ao digitar o objeto

4

Contrato

Define claramente quais dados a API espera receber do front-end

Criando o Service Tarefas

No Angular, a **camada de serviço** é responsável por centralizar toda a comunicação com APIs externas. O **TarefaService** encapsula a chamada HTTP, tornando o componente mais limpo e o código reutilizável em toda a aplicação.

```
import { inject, Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Tarefa } from '../models/tarefas';

@Injectable({
  providedIn: 'root'
})
export class TarefaService {

  private http = inject(HttpClient);
  private api = 'https://jsonplaceholder.typicode.com/posts';
```

@Injectable

Registra o serviço no injetor raiz da aplicação, disponível globalmente

HttpClient

Injetado via construtor para realizar requisições HTTP de forma reativa

Endpoint

URL da API JSONPlaceholder que receberá o payload da nova tarefa

Observable

O método retorna um Observable que o componente deve assinar com `subscribe()`

Consumindo o Service no Componente

Com o serviço pronto, o componente apenas precisa **injetar o Service** e chamar o método `criar()` ao submeter o formulário.

Chamada no submit do formulário

```
criar(tarefa: Tarefa) {  
    return this.http.post<Tarefa>(this.api,  
    tarefa);  
}
```

❗ O método `this.form.value` retorna um objeto com todos os campos do formulário reactive, pronto para ser enviado como payload.

Fluxo de execução

- Submit do formulário
Usuário clica em "Salvar" e o evento é disparado
- Chamada ao service
O componente delega ao service a responsabilidade de comunicação HTTP
- POST enviado
O HttpClient serializa o objeto e envia a requisição ao servidor

Entendendo os Headers HTTP

Os **headers** são metadados que acompanham a requisição HTTP, fornecendo informações contextuais ao servidor. Eles definem o formato dos dados enviados, credenciais de autenticação e outras instruções de processamento.

Exemplo de header no Angular

```
headers: {  
  'Content-Type': 'application/json'  
}
```

- ❏ O Angular HttpClient define automaticamente o `Content-Type: application/json` ao serializar objetos, mas você pode sobrescrever ou adicionar headers customizados via `HttpHeaders`.

Headers mais comuns

Content-Type

Define o formato do corpo da requisição — geralmente `application/json`

Authorization

Carrega token JWT ou credenciais para autenticar o usuário na API

Accept

Indica quais formatos de resposta o cliente aceita do servidor

Custom Headers

Headers personalizados exigidos por APIs específicas, como chave de API

Trabalhando com Payloads Complexos

APIs reais frequentemente exigem payloads mais ricos, com **objetos aninhados** e **arrays**. O Angular serializa automaticamente essas estruturas para JSON, mas é fundamental modelar corretamente as interfaces TypeScript para garantir a tipagem de cada nível.

Exemplo de payload complexo

```
{
  "titulo": "Projeto API",
  "responsavel": {
    "nome": "Jefferson",
    "setor": "TI"
  },
  "tags": ["angular", "api", "frontend"],
  "status": "ativo"
}
```

Conceitos aplicados

Objetos Aninhados

O campo `responsavel` é um objeto dentro do objeto principal, exigindo interface própria no TypeScript

Arrays de Dados

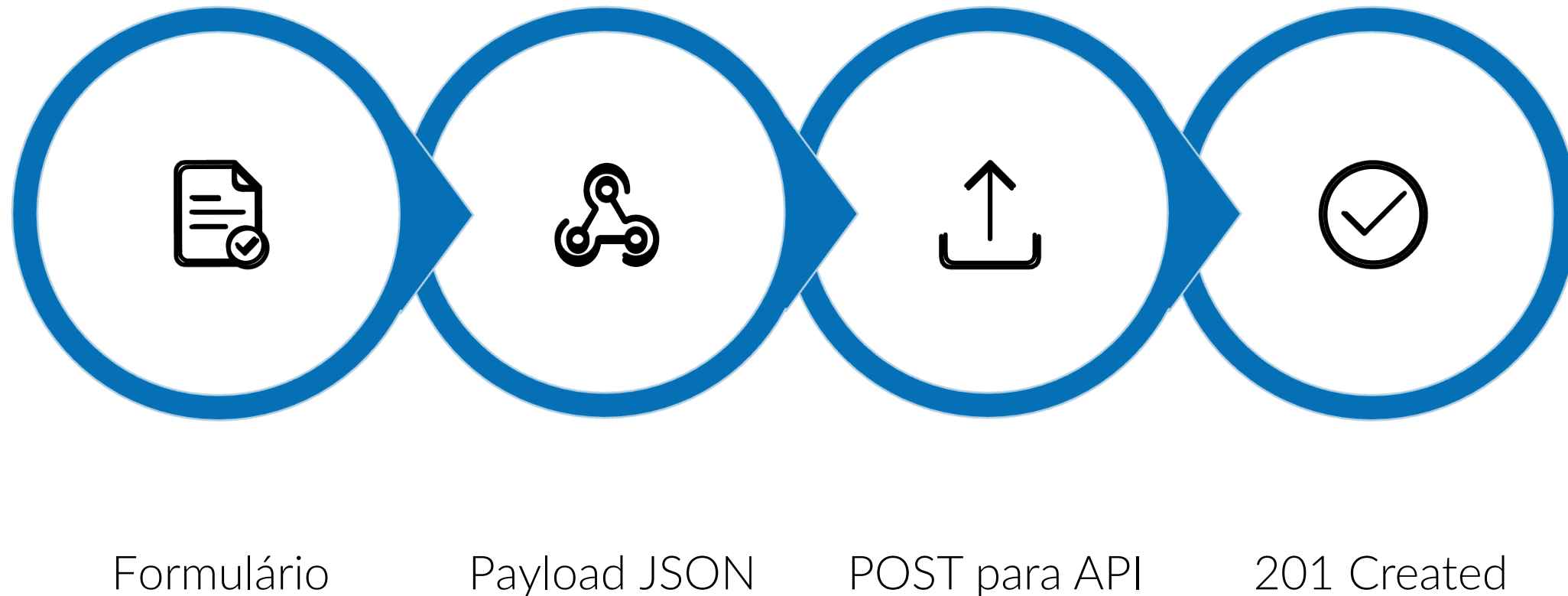
O campo `tags` é uma lista de strings, representando múltiplos valores para um único campo

Mercado Real

Pedidos, projetos, cadastros corporativos — todos usam estruturas complexas como esta no dia a dia

Fluxo Completo

Do Front-end à API: Diagrama da Requisição



Este fluxo representa o ciclo completo de uma operação POST: o usuário interage com o formulário, os dados são serializados em JSON, a requisição é enviada ao servidor e a API confirma a criação do recurso retornando o identificador gerado.

Testando com JSONPlaceholder

A **JSONPlaceholder** é uma API pública gratuita ideal para protótipos e estudos. Ela simula um servidor real, aceitando requisições POST e retornando respostas autênticas com o identificador do recurso criado — sem necessidade de backend próprio.

Payload enviado

```
{
  "title": "Nova tarefa",
  "body": "Descrição da tarefa",
  "userId": 1
}
```

Endpoint: `https://jsonplaceholder.typicode.com/posts`

Resposta esperada — 201 Created

```
{
  "title": "Nova tarefa",
  "body": "Descrição da tarefa",
  "userId": 1,
  "id": 101
}
```

- ✔ O servidor criou o recurso e devolveu o **identificador único** gerado (`id: 101`). Esse ID é fundamental para operações futuras de atualização ou exclusão.

Prática Orientada

Atividade Guiada: Cadastro de Tarefa

Agora é a sua vez! Aplique todos os conceitos da aula implementando um mini projeto completo de cadastro de tarefa com envio via POST.

01

Criar o formulário Angular

Monte o formulário com os campos: título, descrição, prioridade, categoria e botão Salvar usando Reactive Forms

02

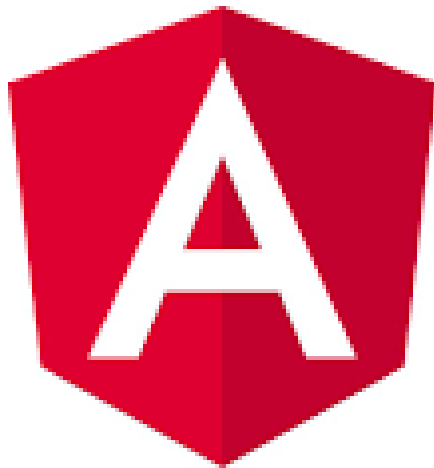
Modelar a interface TypeScript

Crie a interface **Tarefa** com todos os campos necessários para tipagem correta do payload

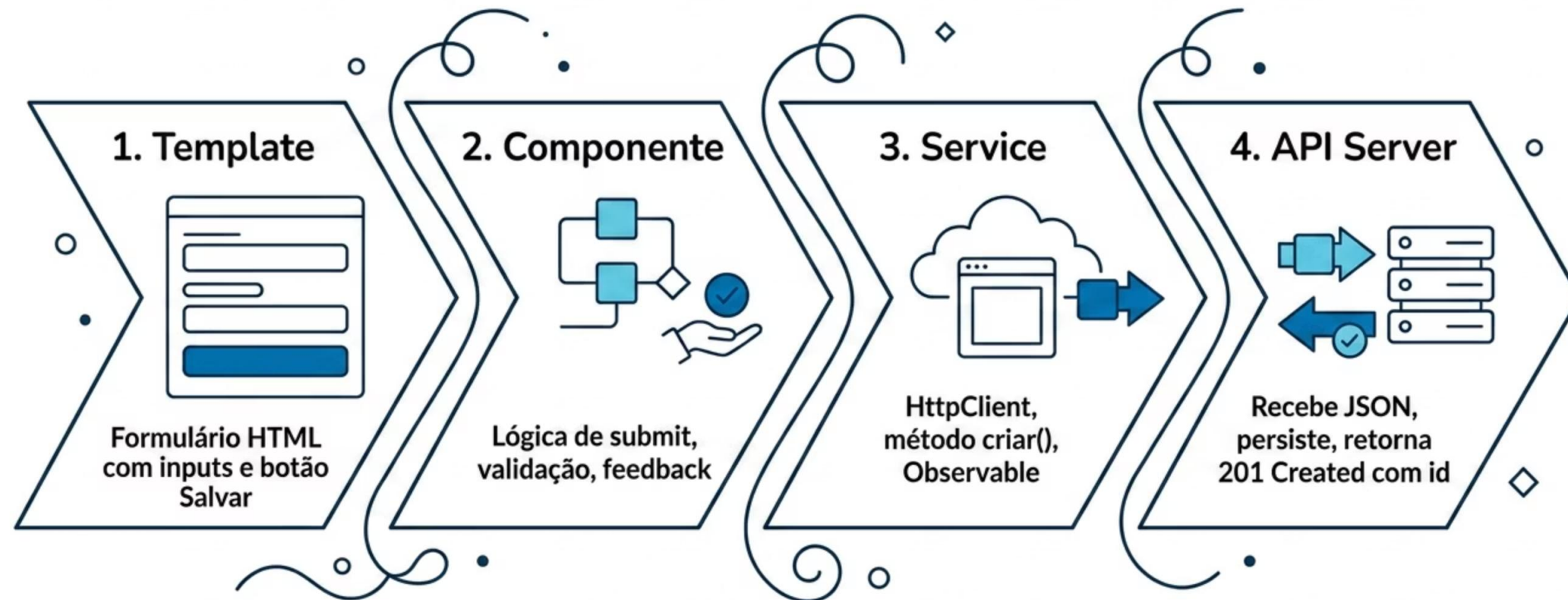
03

Implementar o TarefaService

Crie o service com o método **criar()** usando **HttpClient.post()** apontando para a JSON Placeholder



Recapitulando a Arquitetura



A separação em camadas é um princípio fundamental de arquitetura Angular: o **template** lida com a interface, o **componente** gerencia a lógica, o **service** centraliza a comunicação HTTP e a **API** persiste os dados. Cada camada tem uma responsabilidade clara e única.

Perguntas para Aprofundar o Conhecimento

Bons desenvolvedores não apenas implementam — eles **questionam e preveem cenários**. Reflita sobre estas situações que você encontrará no mercado de trabalho:

 Como validar antes de enviar?

Validar no front com Validators do Angular evita requisições desnecessárias e melhora a experiência do usuário

 Quem gera o ID?

Em APIs REST, o servidor é responsável por gerar e retornar o identificador único do recurso criado

 E se a API rejeitar?

Use o segundo parâmetro do `subscribe()` ou o operador `catchError` para tratar erros HTTP como 400 ou 422

 Payload grande afeta performance?

Sim! Enviar apenas os campos necessários (princípio do payload mínimo) reduz latência e consumo de banda

 Onde montar o objeto?

A montagem do payload deve ocorrer no componente, antes de passar ao service, mantendo o service genérico e reutilizável

Armadilhas que Todo Dev Precisa Evitar

✗ Esquecer o subscribe()

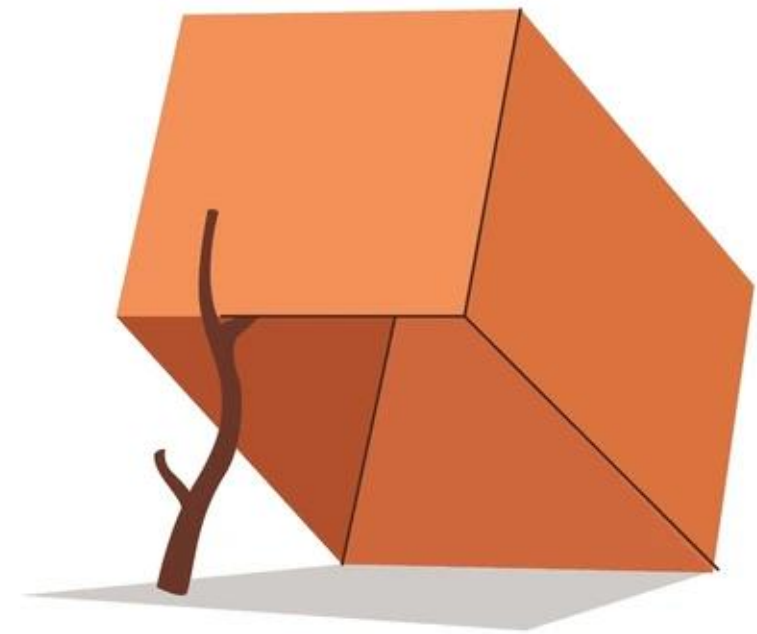
Observables são lazy — sem `subscribe()`, a requisição HTTP **nunca é disparada**. Sempre assine o Observable!

✗ Não importar HttpClientModule

Sem o `HttpClientModule` no `app.module.ts`, o `HttpClient` não estará disponível para injeção de dependência

✗ Campos com nomes errados

O payload deve respeitar exatamente o contrato da API. Um campo com nome errado resulta em erro 400 ou dado ignorado pelo servidor



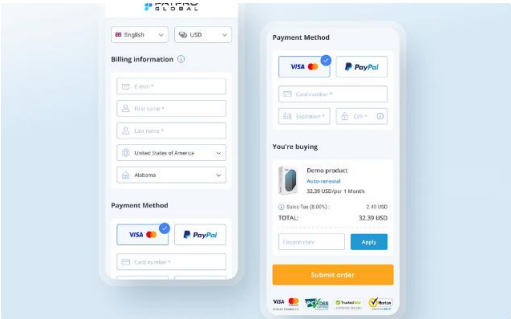
Glossário da Aula

Termos técnicos essenciais que você encontrará na documentação, nas entrevistas de emprego e no dia a dia como desenvolvedor Angular:

Termo	Definição	Exemplo
POST	Método HTTP para criação de recursos no servidor	<code>http.post(url, body)</code>
Payload	Corpo da requisição com os dados a serem enviados	Objeto JSON com campos da tarefa
Header	Metadado que acompanha a requisição HTTP	<code>Content-Type: application/json</code>
Serialização	Conversão de objeto em formato de transporte (JSON)	TypeScript → JSON string
201 Created	Código HTTP que confirma criação bem-sucedida	Resposta do servidor após POST
Observable	Fluxo de dados assíncrono do RxJS usado pelo HttpClient	Retorno de <code>http.post()</code>
Interface	Contrato TypeScript que define a estrutura de um objeto	<code>interface Tarefa { ... }</code>

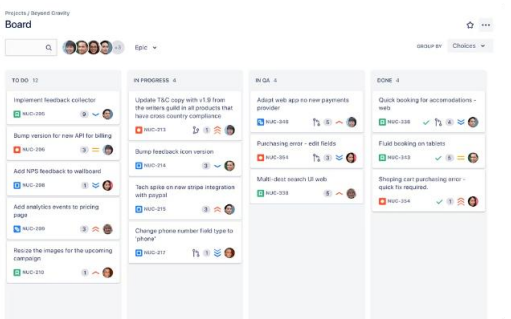
Conexão com o Mercado

POST no Mundo Real



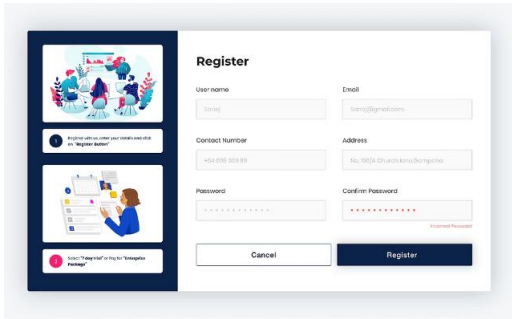
E-commerce

Toda finalização de compra envia um POST com dados do pedido, endereço e pagamento para o backend de processamento



Gestão de Projetos

Ferramentas como Jira e Trello usam POST para criar cards, tarefas e sprints via API — exatamente o que estamos construindo



Autenticação

Cadastro de usuários, redefinição de senha e criação de perfis são todas operações POST com payloads estruturados

O que Você É Capaz de Fazer Agora

Ao concluir esta aula, você demonstrou domínio sobre os fundamentos de criação de dados com Angular e está preparado para construir funcionalidades reais de cadastro em sistemas corporativos.



Criar Payloads JSON
Estruturar objetos simples e complexos para envio via HTTP



Modelar Interfaces TypeScript
Garantir tipagem e segurança de dados em toda a aplicação



Enviar POST com Angular
Usar HttpClient e Services para comunicação com APIs externas



Interpretar Respostas
Compreender status codes e extrair dados retornados pelo servidor



Trabalhar com Headers
Configurar metadados de autenticação e tipo de conteúdo da requisição



Integrar Formulários com APIs
Conectar Reactive Forms ao backend através de services Angular

🎉 **Parabéns!** Você completou a Aula 5 — Manipulação de Dados com APIs. Continue praticando com a atividade guiada e explore a documentação oficial do Angular HttpClient.

